Heterogeneous Expert Architecture

Selection Rationale, Mathematical Formulation,
and Financial Application

Author 1[1], Author 2[1]

[1]*Organization Name*
contact@org.com

Abstract. This document provides a detailed account of the selection rationale, mathematical foundations, and financial domain application of the 7 heterogeneous experts used in the Progressive Layered Extraction (PLE) architecture for financial product recommendation. Each expert offers a unique mathematical perspective—feature interaction (DeepFM), time-series decomposition (Temporal Ensemble), hierarchical structure (Hyperbolic GCN), topological structure (PersLay/TDA), collaborative filtering (LightGCN), causal inference (Causal/NOTEARS), and distributional matching (Optimal Transport)—and no single expert can substitute for any other. Additionally, this document covers the dynamic knowledge transfer mechanism across 18 tasks via adaTT (Adaptive Task-aware Transfer), and the 316-dimensional feature engineering framework derived from 11 academic disciplines.

Keywords: Heterogeneous Experts, PLE, DeepFM, Mamba, Hyperbolic GCN, Persistent Homology, NOTEARS, Optimal Transport, adaTT, Financial AI

Design vs. Implementation Note. This document is written based on the full-bank design (734D). The current Santander benchmark implementation uses 316D (12 feature groups).

1 DeepFM Expert — Feature Interaction

1.1 Selection Rationale (Why DeepFM?)

Financial recommendation depends not on individual features but on feature *interactions*. For example, the pattern where "30s + high digital engagement + high RFM" combine to sharply increase online investment conversion rates cannot be captured by any single feature alone. Explicitly modeling all pairwise interactions for 644 features requires $O(n^2) = 207,046$ parameters, most of which cannot be stably estimated from sparse data.

FM's low-rank factorization reduces the interaction parameters to $O(nk) = 10,304$ while enabling generalization to unobserved feature pairs through shared latent vectors. The Deep component extends beyond FM's 2nd-order limitation by adding implicit higher-order interactions of 3rd order and above.

1.2 Alternative Comparison (Why Not Alternatives?)

Aspect	**DeepFM**	**DCNv2**	**AutoInt (Transformer)**
Interaction order	FM: 2nd + Deep: arbitrary	Cross: $(l+1)$-th + Deep	Self-attention: arbitrary
Parameters	$\sim$ 169K	$\sim$ 2.5M (cross alone)	$O(n^2)$ attention
Inference speed	Fastest (2–5x vs Attention)	Medium	Slowest
Field awareness	Yes (28 fields)	No (raw features)	Yes (per-feature)

Table 1: DeepFM vs. alternative architectures. In BARS benchmarks (2022–2024), DeepFM consistently achieves top-tier performance with the lowest inference latency on small-to-medium datasets.

Unlike Wide&Deep (Google, 2016), DeepFM shares embeddings between FM and Deep, eliminating manual cross-feature engineering. It provides the combination of "structural efficiency (FM) + universal expressiveness (Deep)."

1.3 Key Mathematical Formulations

FM 2nd-order interaction:

$$\hat{y} = w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle x_i x_j \tag{1}$$

Low-rank factorization:

$$W \approx VV^\top, \quad V \in \mathbb{R}^{n \times k} \tag{2}$$

This reduces parameters from $O(n^2)$ to $O(nk)$. The project uses $n = 644$, $k = 16$.

FM Trick ($O(nk)$ computation):

$$\sum_{i<j} \langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle x_i x_j = \frac{1}{2} \sum_{f=1}^{k} \left[\left(\sum_{i=1}^{n} v_{i,f} x_i \right)^2 - \sum_{i=1}^{n} \left(v_{i,f} x_i \right)^2 \right] \tag{3}$$

Cross Network (DCNv2) layer:

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_0 \circ (\boldsymbol{x}_l W_l + \boldsymbol{b}_l) + \boldsymbol{x}_l \tag{4}$$

Each layer adds one order of interaction, and l layers capture up to $(l+1)$-th order interactions. Only $2d$ parameters per layer (compared to $O(d^2)$ for MLP).

1.3.1 28-Field Architecture

644D normalized features are split into 28 semantic fields (base 238D, multi-source 91D, extended 84D, domain 159D, multidisciplinary 24D, model-derived 27D, merchant 21D). Each field is projected into a uniform 16D latent space via `nn.Linear(d_i, 16)`. FM operates on field-level embeddings $[B, 28, 16]$, while Deep MLP operates on the flattened $[B, 448]$.

Output: FM 16D + Deep 64D = 80D $\rightarrow$ output layer $\rightarrow$ 64D (PLE gate input).

1.4 Financial Application

- Cross-field interactions are essential in banking: "30s + Seoul + high digital engagement" $\rightarrow$ online investment conversion rate surge; "high RFM + low deposits" $\rightarrow$ credit product recommendation.
- The 28-field design enables inter-category FM interaction: splitting a 64D category into 4×16D creates 27 new FM interaction pairs at no additional parameter cost.

1.5 I/O Specification

Input	644D normalized feature tensor $[B, 644]$
Internal	28 fields $\times$ 16D embeddings $\rightarrow$ FM $[B, 28, 16]$ + Deep $[B, 448]$
Output	64D expert representation for PLE CGC Gate
Parameters	$\sim$ 169K

Table 2: DeepFM Expert I/O specification.

1.6 Implementation Notes

- Used as PLE's default lightweight Shared Expert. DCNv2Expert is an alternative for cases requiring higher-order interactions.
- Since FM and Deep share embeddings, gradients flow simultaneously through both paths.
- When applying FocalLoss, pre-activation logits must be passed (to prevent double-sigmoid).

Key References: Guo et al. (IJCAI 2017), Rendle (ICDM 2010), Wang et al. (KDD 2017, WWW 2021).

2 Temporal Ensemble Expert — Mamba + LNN + PatchTST

2.1 Selection Rationale (Why Temporal Ensemble?)

Static features (age, average spending) discard the temporal dimension of transaction sequences—periodicity, trends, and irregular event patterns. Compressing a 180-day spending sequence into a monthly average loses weekly cycles, trend directions, and anomalous burst patterns.

All time series decompose as $y(t) = T(t) + S(t) + R(t)$ (trend + seasonality + residual), and no single architecture can optimally capture all three:

Model	Temporal Pattern	Mechanism	Complexity
Mamba (SSM)	Long-range trend	Selective State Space (S6)	$O(L)$ linear
LNN (ODE)	Irregular residuals	Adaptive time-constant ODE	$O(1)$ single step
PatchTST	Global periodicity	Patch-level self-attention	$O((L/P)^2)$

Table 3: The three models of the Temporal Ensemble – corresponding to the time-series decomposition $T + S + R$.

2.2 Alternative Comparison (Why Not Single Model?)

Generation	Approach	Limitation
1st	ARIMA, Exponential Smoothing	Linear assumption, manual differencing
2nd	LSTM, GRU	$O(L)$ sequential bottleneck, vanishing gradient
3rd	Transformer	$O(L^2)$ complexity, weak order encoding
4th (ours)	SSM + ODE + Patch Transformer	Model complexity (mitigated by entropy monitoring)

Table 4: Comparison by time-series modeling generation.

2.3 Key Mathematical Formulations

2.3.1 Mamba (Selective State Space Model)

Continuous SSM:

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}u, \quad y = \boldsymbol{C}\boldsymbol{x} + \boldsymbol{D}u \tag{5}$$

ZOH Discretization:

$$\bar{\boldsymbol{A}} = \exp(\Delta \cdot \boldsymbol{A}), \quad \bar{\boldsymbol{B}} \approx \Delta \cdot \boldsymbol{B} \tag{6}$$

Discrete recurrence:

$$\boldsymbol{h}_t = \bar{\boldsymbol{A}} \cdot \boldsymbol{h}_{t-1} + \bar{\boldsymbol{B}} \cdot \boldsymbol{x}_t, \quad \boldsymbol{y}_t = \boldsymbol{C}_t \cdot \boldsymbol{h}_t \tag{7}$$

S6 Selective Mechanism (key innovation):

$$\Delta = \text{softplus}(\boldsymbol{W}_\Delta \cdot \boldsymbol{x} + \boldsymbol{b}_\Delta) \tag{8}$$

$$\boldsymbol{B} = \boldsymbol{W}_B \cdot \boldsymbol{x}, \quad \boldsymbol{C} = \boldsymbol{W}_C \cdot \boldsymbol{x} \tag{9}$$

Unlike LTI systems, S6 makes Δ, $\boldsymbol{B}$, $\boldsymbol{C}$ *input-dependent*, enabling content-aware processing. A large Δ strongly encodes the current input into state; a small Δ preserves the previous state.

Financial interpretation: Large transactions → large Δ (strongly remembered), small routine purchases → small Δ (quickly forgotten as background noise).

Specification: Transaction Mamba: $d_{\text{model}} = 128$, $d_{\text{input}} = 16$, $d_{\text{state}} = 16$, seq_len $= 180$. Session Mamba: $d_{\text{model}} = 64$. HiPPO-style diagonal initialization $[-1, -2, ..., -N]$.

2.3.2 Liquid Neural Network (LNN)

Core ODE:

$$\frac{d\boldsymbol{h}}{dt} = \frac{-\boldsymbol{h} + f(\boldsymbol{x}, \boldsymbol{h})}{\tau(\boldsymbol{x}, \boldsymbol{h})} \tag{10}$$

Where:
- $-\boldsymbol{h}$: leak/decay term (forgets toward 0 without input)
- $f(\boldsymbol{x}, \boldsymbol{h}) = \tanh\big(\boldsymbol{W}_f[\boldsymbol{x}; \boldsymbol{h}] + \boldsymbol{b}_f\big)$: target state
- $\tau(\boldsymbol{x}, \boldsymbol{h}) = \text{Softplus}(\text{MLP}([\boldsymbol{x}; \boldsymbol{h}])) + 0.1$: adaptive time constant

Euler discretization:

$$\boldsymbol{h}_{t+1} = \boldsymbol{h}_t + \Delta t \cdot \frac{-\boldsymbol{h}_t + f(\boldsymbol{x}_t, \boldsymbol{h}_t)}{\tau(\boldsymbol{x}_t, \boldsymbol{h}_t)} \tag{11}$$

The project uses SingleStep mode — applying only 1 ODE step to Mamba's final hidden state. Since Mamba already captures the full sequence pattern at $O(L)$, LNN adds only *time-scale correction.*

Financial interpretation: Financial transaction intervals are extremely irregular (multiple intraday trades: $\Delta t \sim 0.01$ days, weekend gaps: $\Delta t = 2$ days, long dormancy: $\Delta t > 30$ days). Adaptive τ adjusts automatically.

2.3.3 PatchTST (Patch Time Series Transformer)

Patch embedding:

$$\boldsymbol{p}_i = \boldsymbol{W}_{\text{proj}} \cdot \text{flatten}\big(\boldsymbol{x}_{[[(i-1)P+1:iP]]}\big) + \boldsymbol{b}_{\text{proj}} \tag{12}$$

With $P = 16$, a 180-step sequence becomes 12 patches (tokens), reducing attention cost from $O(180^2)$ to $O(12^2) = 144$.

Financial interpretation: Patch size 16 corresponds to approximately 2 weeks, naturally aligning with salary cycles (biweekly/monthly). Each patch captures local patterns (daily spending within 2 weeks), while inter-patch attention captures global periodicity (payday surges, quarterly bonuses).

2.3.4 Ensemble Gating

$$\boldsymbol{g} = \text{Softmax}(\boldsymbol{W}_2 \cdot \text{ReLU}(\boldsymbol{W}_1 \cdot \boldsymbol{z}_{\text{cat}} + \boldsymbol{b}_1) + \boldsymbol{b}_2) \in \mathbb{R}^3 \tag{13}$$

$$\boldsymbol{y} = \sum_{i=1}^{3} g_i \cdot \text{Proj}_i(\boldsymbol{z}_i) \in \mathbb{R}^{64} \tag{14}$$

$\boldsymbol{z}_{\text{cat}}$ is the concatenation of the three model outputs (192+96+96 = 384D). Gate entropy $H(\boldsymbol{g}) = -\sum_{i=1}^{3} g_i \log_2(g_i)$ is monitored, and gate collapse is detected when $H < 0.3$ bits.

2.4 Financial Application

Financial transactions exhibit strong periodicity (payday, weekend dining), gradual trends (lifestyle changes, approaching churn), and irregular residuals (travel, fraud). Each component maps directly to a model's strength: Mamba → trends, PatchTST → seasonality, LNN → residuals.

2.5 I/O Specification

Input	Transaction seq $[B, 180, 16]$ + Session seq $[B, 90, 8]$
Mamba output	128D + 64D = 192D
LNN output	64D + 32D = 96D (SingleStep on Mamba final state)
PatchTST output	64D + 32D = 96D
Ensemble output	64D (gated combination for PLE gate)

Table 5: Temporal Ensemble Expert I/O specification.

2.6 Implementation Notes

- Mamba → LNN is serial (LNN corrects time-scale after Mamba training), PatchTST is independent. This is a design for ensuring ensemble diversity through input separation.
- The A matrix is initialized with HiPPO-style diagonal $[-1, -2, ..., -N]$ to implement multi-scale memory decay.
- A gate collapse warning is triggered when gate entropy drops from $\log_2(3) \approx 1.585$ bits (uniform distribution) to < 0.3 bits, indicating one model is dominating.

Key References: Gu & Dao (NeurIPS 2023), Hasani et al. (AAAI 2021), Nie et al. (ICLR 2023).

3 Hyperbolic GCN Expert — Hierarchical Structure

3.1 Selection Rationale (Why Hyperbolic GCN?)

Recommendation data has two fundamentally different geometric structures: (1) user-item interactions are *peer-to-peer* (no hierarchy), (2) Merchant Category Codes (MCC) form a *tree hierarchy* (Root → L1 → L2 → Brand → Branch, ∼550K nodes). A single geometry cannot efficiently represent both.

Embedding a complete binary tree of depth d with uniform spacing in Euclidean space requires $O(2^d)$ dimensions. Distortion-free embedding of ∼50K brand-level nodes would require tens of thousands of dimensions. Hyperbolic space (negative curvature) has volume that increases *exponentially* away from the origin, exactly matching tree branching. **An 8D Poincare Ball can represent the entire MCC hierarchy.**

Nickel & Kiela (2017) showed that 5D hyperbolic embeddings outperform 200D Euclidean embeddings on the WordNet hierarchy.

3.2 Alternative Comparison

- **LightGCN vs. NGCF:** Removing feature transformation W and nonlinear activation σ actually *improves* performance in ID-based collaborative filtering. When there are no raw features to transform, simpler is better.
- **H-GCN vs. Euclidean tree embedding:** A complete binary tree of depth d requires $O(2^d)$ dimensions in Euclidean space but only $O(d)$ dimensions in hyperbolic space.
- **2-Stage vs. end-to-end GCN:** Pinterest PinSage pattern — offline graph precomputation is the standard for production systems. It decouples graph update frequency from model training frequency.

3.3 Mathematical Background

3.3.1 Poincare Ball Model (H-GCN)

$$\mathbb{B}_c^d = \{\boldsymbol{x} \in \mathbb{R}^d : c\,\|\boldsymbol{x}\|^2 < 1\} \tag{15}$$

Exponential map (tangent → hyperbolic):

$$\exp_{\mathbf{0}}(\boldsymbol{v}) = \tanh\!\big(\sqrt{c}\,\|\boldsymbol{v}\|\big) \cdot \frac{\boldsymbol{v}}{\sqrt{c}\,\|\boldsymbol{v}\|} \tag{16}$$

Logarithmic map (hyperbolic → tangent):

$$\log_{\mathbf{0}}(\boldsymbol{y}) = \operatorname{arctanh}\!\big(\sqrt{c}\,\|\boldsymbol{y}\|\big) \cdot \frac{\boldsymbol{y}}{\sqrt{c}\,\|\boldsymbol{y}\|} \tag{17}$$

Poincare distance:

$$d_{\mathbb{B}}(\boldsymbol{x},\boldsymbol{y}) = \frac{1}{\sqrt{c}}\,\operatorname{arccosh}\left(1 + \frac{2c\,\|\boldsymbol{x}-\boldsymbol{y}\|^2}{(1-c\|\boldsymbol{x}\|^2)(1-c\|\boldsymbol{y}\|^2)}\right) \tag{18}$$

Near the boundary: the denominator → 0, so distance explodes (small Euclidean distance = large hyperbolic distance). This naturally encodes hierarchical depth.

Riemannian gradient correction:

$$\nabla_{\text{Riem}} f(\boldsymbol{x}) = \frac{\big(1-c\|\boldsymbol{x}\|^2\big)^2}{4}\nabla_{\text{Euclid}} f(\boldsymbol{x}) \tag{19}$$

Fermi-Dirac decoder (link prediction):

$$P(\text{edge} \mid u, v) = \frac{1}{\exp((d_{\mathbb{B}}(u,v) - r)/t) + 1} \tag{20}$$

Borrowed from statistical physics. r = margin (Fermi energy), t = temperature.

Frechet mean (Einstein midpoint approximation):

$$\gamma_i = \frac{1}{\sqrt{1 - c\|\boldsymbol{x}_i\|^2}}, \quad \bar{\boldsymbol{x}} = \text{proj}\left(\frac{\sum_i w_i \gamma_i \boldsymbol{x}_i}{\sum_i w_i \gamma_i}\right) \tag{21}$$

The Lorentz factor γ_i assigns higher weight to boundary points (specialized consumers).

3.4 Financial Application

- **LightGCN**: Captures indirect preferences through multi-hop collaborative signals. "Customer A purchases at Starbucks, Customer B purchases at both Starbucks and Ediya" → A may prefer Ediya.
- **H-GCN**: The MCC classification hierarchy (Root → L1(8) → L2(∼100) → Brand(∼50K) → Branch(∼500K)) is inherently a tree structure.
- **Co-visitation edges**: Behavioral signals complement the static MCC hierarchy. "Starbucks visitors who also visit Ediya within 7 days" → edges weighted with exponential time decay.

3.5 I/O Specification

Property	**LightGCN**	**H-GCN**
Nodes	Customers + Merchants (bipartite)	Merchants only (MCC tree)
Edges	Customer-Merchant transactions	Parent-child + Brand co-visitation
Space	Euclidean $\mathbb{R}^{64}$	Poincare Ball $\mathbb{B}^8$
Learning	"Who likes what" (CF)	"How merchants relate" (hierarchy)
Output	Customer embedding 64D (direct)	Merchant emb → per-customer agg 47D

Table 6: Dual GCN architecture comparison.

3.6 Implementation Notes

- **2-Stage pipeline:** Stage 1 (offline) — graph-level training (LightGCN: BPR, H-GCN: self-supervised Fermi-Dirac). Embeddings stored as Parquet. Stage 2 (online) — lookup + lightweight MLP adaptation. No graph propagation at inference — single GPU VRAM friendly.
- Co-visitation edge scale factor of 0.5 is applied to preserve the structural priority of taxonomy edges.

Key References: He et al. (SIGIR 2020), Chami et al. (NeurIPS 2019), Nickel & Kiela (NeurIPS 2017).

4 PersLay / TDA Expert — Topological Structure

4.1 Selection Rationale (Why TDA/PersLay?)

Traditional statistics (mean, variance, correlation) summarize distribution moments but miss the data's *structural shape* — cluster connectivity, cyclic consumption patterns, voids in the spending space. Two customers with identical mean/variance can have fundamentally different consumption topologies (one continuous cluster vs. two separated clusters with periodic switching).

Persistent Homology observes data simultaneously at all scales (filtration), tracking when topological features (connected components H_0, loops H_1, voids H_2) appear and disappear. It provides three unique properties: (1) coordinate invariance, (2) multi-resolution analysis without single threshold selection, (3) mathematically guaranteed noise robustness via the stability theorem.

4.2 Alternative Comparison

Method	Variable-size PD	End-to-end learnable	Stability guarantee
Persistence Images	Fixed grid	No	Approximate
Persistence Landscapes	Function space	No (Banach space)	Yes
PersLay	**Yes (phi + rho)**	**Yes**	**Yes (inherits)**
Persformer (2024)	Yes (attention)	Yes	Yes

Table 7: PD processing method comparison. PersLay has no gradient bottleneck with sum aggregation and handles padding automatically via persistence weighting, ensuring high production stability.

4.3 Mathematical Background

4.3.1 PersLay Layer (Carriere et al., 2020)

$$\mathrm{PersLay}(D) = \rho\left(\sum_{(b,d)\in D} w(b,d)\cdot\varphi(b,d)\right) \tag{22}$$

- φ: point transformation (RationalHat or Gaussian)
- w: weighting function (persistence-based: $w = |d-b|^p$, or learned)
- ρ: permutation-invariant aggregation (sum, mean, max, or attention)

4.3.2 Homology Groups

$$H_k = \mathrm{Ker}(\partial_k)/\ \mathrm{Im}(\partial_{k+1}) \tag{23}$$

$\beta_k = \mathrm{rank}(H_k)$: the number of independent k-dimensional "holes." β_0 = connected components, β_1 = loops, β_2 = voids.

4.3.3 Stability Theorem

$$d_B(\mathrm{Dgm}(f), \mathrm{Dgm}(g)) \le \|f-g\|_\infty \tag{24}$$

Input perturbation is bounded by the maximum change in the filtration function. This mathematically guarantees noise robustness of topological features.

4.3.4 Vietoris-Rips Complex

$$\sigma = \{x_0, ..., x_k\} \in \mathrm{VR}_\varepsilon(X) \Longleftrightarrow d(x_i, x_j) \le \varepsilon, \quad \forall i,j \tag{25}$$

4.3.5 Persistence Entropy (Rucco et al., 2016)

$$E = -\sum_{i=1}^{N} p_i \log p_i, \quad p_i = \frac{d_i - b_i}{\sum_j (d_j - b_j)} \tag{26}$$

High entropy = diverse topological features uniformly distributed. Low entropy = a few dominant patterns.

4.3.6 Wasserstein-1 Distance for Phase Transition

$$W_1(\mathrm{PD}_1, \mathrm{PD}_2) = \inf_{\gamma} \sum_{x \in \mathrm{PD}_1} \|x - \gamma(x)\|_\infty \tag{27}$$

Measures structural change between persistence diagrams of the first-half and second-half transactions.

Phase transition probability:

$$P_{\text{transition}} = \frac{1}{1 + e^{-2(\Delta_{\text{total}} - \tau)}}, \quad \tau = 0.5 \tag{28}$$

4.3.7 5-Block Multi-Beta Architecture

Block	Input	Homology	Output
short_beta0	90-day app logs $[B, 200, 3]$	H_0 (clusters)	64D
short_beta1	90-day app logs	H_1 (loops)	64D
long_beta0	12-month txn $[B, 150, 3]$	H_0 (clusters)	64D
long_beta1	12-month transactions	H_1 (loops)	64D
long_beta2	12-month transactions	H_2 (voids)	64D

Table 8: PersLay 5-block configuration.

Short concat 128D + Long concat 192D + Global stats MLP 32D + Phase transition 10D = **362D → final_mlp → 64D**.

4.3.8 TDA Offline Feature Pipeline (70D)

70D = 24D (short) + 36D (long) + 10D (phase):

- **tda_short (24D):** 90-day app logs. 6D point cloud per transaction. $H_0 + H_1 \times 6$ features (entropy, lifetime mean/std/min/max/median) $\times$ 2 scopes.
- **tda_long (36D):** 12-month card transactions. $H_0 + H_1 + H_2 \times 6$ features $\times$ 2 scopes.
- **phase_transition (10D):** PD Distance 4D + Transition Detection 6D.

4.4 Financial Application

- **H_0 (connected components):** Separated consumption clusters reveal lifestyle segmentation. The merging of "grocery cluster" vs. "travel cluster" at different scales indicates spending diversification.
- **H_1 (loops):** Cyclic consumption patterns (monthly: grocery → transport → entertainment → grocery) are captured as persistent loops. Strong loops = habitual patterns = predictable behavior.
- **H_2 (voids):** 3D voids in the amount-category-time space reveal systematic avoidance patterns (e.g., absence of mid-range spending = small + large amounts only).
- **Phase transition:** Wasserstein distance quantifies behavioral change (job change, life events, financial crisis).

4.5 I/O Specification

PersLay Input	Raw persistence diagrams (dual mode: raw PD / pre-computed 70D fallback)
PersLay Output	64D expert representation for PLE gate
TDA Offline Output	70D features integrated into main 734D tensor
Computation	Ripser++ (GPU) → Ripser (CPU) → giotto-tda (fallback)

Table 9: PersLay / TDA Expert I/O specification.

4.6 Implementation Notes

- Production config: RationalHatPhi + persistence weighting + sum aggregation.
- Cold-start 4-stage progressive TDA: Day 0 (18D median) → 7–30d (9D histogram) → <12m (24D H0,H1) → 12m+ (36D full).
- Time-stratified sampling: up to 1000 points per customer, stratified across k time buckets to preserve temporal ordering.

Key References: Carriere et al. (AISTATS 2020), Cohen-Steiner et al. (DCG 2007), Bauer (2021).

5 LightGCN Expert — Collaborative Filtering

5.1 Selection Rationale (Why LightGCN?)

In recommendation systems, the user-item interaction graph has a *peer-to-peer* structure (no hierarchy). The key question in this structure is "who likes what," and multi-hop collaborative signals are needed to infer indirect preferences for items without direct interaction.

LightGCN is an architecture that *removes* feature transformation W and nonlinear activation σ from NGCF, actually improving performance in ID-based collaborative filtering. It follows the principle that simpler is better when there are no raw features to transform.

5.2 Alternative Comparison

- **NGCF:** Includes feature transformation + nonlinearity. Unnecessary complexity for collaborative filtering.
- **GraphSAGE:** Neighbor sampling-based, suitable for inductive learning, but LightGCN's full-neighbor utilization is superior in transductive settings.
- **GAT:** Attention mechanism increases overfitting risk with marginal gains relative to cost.

5.3 Mathematical Background

5.3.1 Message Passing

$$\boldsymbol{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|} \cdot \sqrt{|\mathcal{N}_i|}} \cdot \boldsymbol{e}_i^{(k)} \tag{29}$$

Symmetric normalization $\tilde{A} = D^{-1/2} A D^{-1/2}$ attenuates both sender (popular item) and receiver influence.

Layer combination:

$$\boldsymbol{e}_u^{\text{final}} = \frac{1}{L+1} \sum_{k=0}^{L} \boldsymbol{e}_u^{(k)} \tag{30}$$

Uniform averaging over all hops (0-hop self + 1,2,3-hop neighbor). Empirically strong without learnable attention weights, preventing overfitting.

BPR Loss:

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i^+,i^-)} \log \sigma(\hat{y}_{ui^+} - \hat{y}_{ui^-}) + \lambda \, \|\Theta\|^2 \tag{31}$$

5.4 Financial Application

- **Cross-selling:** "Customer A purchases at Starbucks, Customer B purchases at both Starbucks and Ediya" → A may prefer Ediya. Essential for cross-selling in banking.
- **Cold-start mitigation:** Even sparse users can leverage rich signals from similar users through multi-hop propagation.
- **Scalability:** At inference, pre-computed embeddings are looked up without graph propagation, suitable for real-time serving.

5.5 I/O Specification

Input	User-item bipartite graph (customer-merchant transactions)
Embedding dim	64D (Euclidean $\mathbb{R}^{64}$)
Layers	3 hops with uniform averaging
Loss	BPR (pairwise ranking)
Output	Customer embedding 64D for PLE gate

Table 10: LightGCN Expert I/O specification.

5.6 Implementation Notes

- In Stage 1 of the 2-stage pipeline, BPR training is performed offline and embeddings are stored as Parquet.
- L2 regularization is applied only to initial embeddings, not to GCN outputs.
- Separated as a *distinct* expert from H-GCN to ensure independent gradient flows for Euclidean (CF) and hyperbolic (hierarchy) geometries.

Key References: He et al. (SIGIR 2020), Rendle et al. (UAI 2009), Kipf & Welling (ICLR 2017).

6 Causal Expert (NOTEARS) — Causal Inference

6.1 Selection Rationale (Why Causal Expert?)

Standard recommendation systems rely on *correlation* ("customers who bought A also bought B"), conflating spurious association with true causal effects. Example: "Premium cardholders have high travel insurance adoption" may be due to income as a confounder — the card does not *cause* insurance adoption.

A/B testing is the gold standard but does not scale (18 tasks $\times$ N strategies = infeasible), is slow (weeks), and provides only population-level ATE.

6.2 Alternative Comparison

- **A/B Testing:** Gold standard but does not scale, no individual-level ITE.
- **GES/PC Algorithm:** Frequentist causal structure learning but statistical power issues with conditional independence tests.
- **DoWhy/EconML:** Specialized for causal effect estimation but requires separate DAG learning.
- **DAGMA (Bello et al., 2022):** Improved NOTEARS variant, a candidate for future replacement.

6.3 Mathematical Background

6.3.1 Feature Compression

$$\boldsymbol{z} = \text{Compressor}(\boldsymbol{x}) : \mathbb{R}^{644} \to \mathbb{R}^{128} \to \mathbb{R}^{32} \tag{32}$$

Reduces 644D normalized features to 32 causal variables. Prevents the DAG adjacency matrix from exploding to $644^2 \approx 410$K entries.

6.3.2 SCM (Structural Causal Model) Intervention

$$\hat{\boldsymbol{z}} = \boldsymbol{z} + \boldsymbol{z}(\boldsymbol{W} \circ \boldsymbol{W}) \tag{33}$$

- $\boldsymbol{W} \in \mathbb{R}^{32\times 32}$: learnable weighted adjacency matrix (`nn.Parameter`)
- $\boldsymbol{W} \circ \boldsymbol{W}$: element-wise square $\to$ ensures *non-negative* causal strength
- $W_{i,j}^2$: causal influence strength from variable j to i
- Residual connection ($\boldsymbol{z} +$) preserves original information while adding causal adjustment

6.3.3 NOTEARS Acyclicity Constraint

$$h(\boldsymbol{W}) = \text{tr}(e^{\boldsymbol{W} \circ \boldsymbol{W}}) - d = 0 \tag{34}$$

Mathematical interpretation: The (i, i) diagonal element of $e^{\boldsymbol{M}}$ is the sum of all weighted paths from node i back to itself. If the graph is a DAG (acyclic), no such return paths exist, so $e^{\boldsymbol{M}}_{i,i} = 1$ (only the identity matrix contribution), and $\text{tr}(e^{\boldsymbol{M}}) = d$.

Taylor 10-term approximation:

$$e^{\boldsymbol{M}} \approx \sum_{k=0}^{9} \frac{\boldsymbol{M}^k}{k!} \tag{35}$$

Detects cycles up to length 10. Cycles longer than 10 hops are practically impossible in a 32-node DAG.

6.3.4 DAG Regularization Loss

$$\mathcal{L}_{\text{DAG}} = \lambda_{\text{acyclic}} \cdot h(\boldsymbol{W}) + \lambda_{\text{sparse}} \cdot \|\boldsymbol{W} \circ \boldsymbol{W}\|_1 \tag{36}$$

Hyperparameter	Default	Role
`dag_lambda`	0.01	Acyclicity constraint strength
`sparsity_lambda`	0.001	Edge sparsity (L1 on adjacency)
`n_causal_vars`	32	Number of causal nodes

Table 11: Causal Expert hyperparameters. When `dag_lambda > 0.1`, W collapses to the zero matrix (DAG penalty dominates task loss, Expert degenerates to the identity function).

6.4 Financial Application

- Removes confounders (income) and learns *directional* causal relationships to answer "Will this recommendation *cause* behavioral change?"
- The directionality of the SCM ($W_{i,j} \neq W_{j,i}$) provides explainable recommendation paths.
- Enables individual treatment effect (ITE) estimation from observational data.

6.5 I/O Specification

Input	644D normalized features $[B, 644]$
Compressor	$644 \to 128 \to 32$ (causal variables)
SCM	32×32 learnable adjacency $\boldsymbol{W}$
Output	64D causal representation + DAG (visualization)
Auxiliary loss	$\mathcal{L}_{\text{DAG}} = \lambda_{\text{acyclic}} \cdot h(\boldsymbol{W}) + \lambda_{\text{sparse}} \cdot \lVert \boldsymbol{W}^2 \rVert_1$

Table 12: Causal Expert I/O specification.

6.6 Implementation Notes

- The NOTEARS paper uses Augmented Lagrangian (strict equality), but our implementation uses the simple penalty method. It is more stable in end-to-end MTL joint training environments.
- The Hadamard square ($\boldsymbol{W} \circ \boldsymbol{W}$) of the adjacency matrix $\boldsymbol{W}$ enforces non-negative causal strength, unlike the sign-agnostic original.
- Maintained as a *separate* expert from OT Expert: the NOTEARS acyclicity constraint and Sinkhorn entropy regularization have completely different loss surface geometries.

Key References: Zheng et al. (NeurIPS 2018), Bello et al. (ICML 2022).

7 Optimal Transport Expert — Distributional Matching

7.1 Selection Rationale (Why Optimal Transport?)

When comparing a customer's consumption pattern to prototype profiles, the geometric structure of the feature space must be respected. KL divergence is undefined when distribution supports do not overlap, and Euclidean distance ignores the geometric structure of the feature space. Wasserstein distance reflects the *geometry* of the ground metric space — it captures that "Seoul-Incheon" is closer than "Seoul-Busan" even when probability masses do not overlap.

Complementary to the Causal Expert: Causal answers "Will this recommendation *cause* behavioral change?" (directionality), while OT answers "How *close* is this customer's spending distribution to the target profile?" (geometric distance).

7.2 Alternative Comparison

- **KL Divergence:** Undefined when supports do not overlap. Asymmetric.
- **Total Variation:** Ignores ground metric, compares only distribution shape.
- **Euclidean distance:** Ignores semantic structure of the feature space.
- **Sliced Wasserstein:** Computationally efficient alternative, can be independently swapped in the future.

7.3 Mathematical Background

7.3.1 Distribution Projection

$$\boldsymbol{\mu} = \text{softmax}(\text{DistProjector}(\boldsymbol{x})) \in \Delta^{32} \tag{37}$$

Transforms 644D features into a probability simplex, representing each customer's feature profile as a discrete distribution over 32 latent categories.

7.3.2 Learnable Reference Distributions

$$\boldsymbol{\nu}_k = \text{softmax}(\ell_k) \in \Delta^{32}, \quad k = 1, ..., 16 \tag{38}$$

16 learnable prototype customer profiles (`nn.Parameter`), initialized with `randn(16, 32) * 0.1`.

7.3.3 PSD Cost Matrix

$$\boldsymbol{C} = \boldsymbol{M}^\top \boldsymbol{M} \in \mathbb{R}^{32\times 32} \tag{39}$$

Positive semi-definiteness (PSD) guarantee: $\boldsymbol{x}^\top(\boldsymbol{M}^\top\boldsymbol{M})\boldsymbol{x} = \|\boldsymbol{M}\boldsymbol{x}\|^2 \geq 0$. The cost matrix is *learnable*, providing task-optimized semantic distances.

7.3.4 Entropy-Regularized Optimal Transport

Kantorovich problem with entropic regularization:

$$\min_{\boldsymbol{P}\in\mathcal{U}(\boldsymbol{\mu},\boldsymbol{\nu})} \langle \boldsymbol{P}, \boldsymbol{C}\rangle + \varepsilon \cdot H(\boldsymbol{P}) \tag{40}$$

Where:
- $\mathcal{U}(\boldsymbol{\mu},\boldsymbol{\nu}) = \{\boldsymbol{P} \geq 0 : \boldsymbol{P}\mathbf{1} = \boldsymbol{\mu}, \boldsymbol{P}^\top\mathbf{1} = \boldsymbol{\nu}\}$
- $H(\boldsymbol{P}) = -\sum_{i,j} P_{i,j} \log P_{i,j}$: entropy regularization
- $\varepsilon = 0.1$: regularization coefficient

Entropy regularization makes the problem **strictly convex** (unique solution, guaranteed convergence).

7.3.5 Log-Domain Sinkhorn Algorithm

$$\boldsymbol{u}_{\text{new}} = \log \boldsymbol{\mu} - \text{logsumexp}\left(-\frac{\boldsymbol{C}}{\varepsilon} + \boldsymbol{v}\right) \tag{41}$$

$$\boldsymbol{v}_{\text{new}} = \log \boldsymbol{\nu} - \text{logsumexp}\left(-\frac{\boldsymbol{C}^\top}{\varepsilon} + \boldsymbol{u}\right) \tag{42}$$

Log-domain computation prevents floating-point underflow when ε is small. 10 iterations are sufficient for practical convergence.

7.3.6 Wasserstein Distance Vector

$$\boldsymbol{w} = [W(\boldsymbol{\mu}, \boldsymbol{\nu}_1), W(\boldsymbol{\mu}, \boldsymbol{\nu}_2), ..., W(\boldsymbol{\mu}, \boldsymbol{\nu}_{16})] \in \mathbb{R}^{16} \tag{43}$$

Where $W(\boldsymbol{\mu}, \boldsymbol{\nu}_k) = \langle \boldsymbol{P}, \boldsymbol{C} \rangle_F = \sum_{i,j} P_{i,j} \cdot C_{i,j}$.

This creates a **distributional coordinate system** — each customer is positioned by their distances to 16 reference prototypes.

7.3.7 Wasserstein Encoder

$$\boldsymbol{o} = \text{WassersteinEncoder}(\boldsymbol{w}) : \mathbb{R}^{16} \to \mathbb{R}^{128} \to \mathbb{R}^{64} \tag{44}$$

7.4 Financial Application

Wasserstein distance quantifies "How different is this customer's spending pattern from the typical travel/savings/dining profile, and *in which categories and in which direction* must it shift to match?"

This provides directional information impossible with KL divergence or Euclidean distance.

7.5 I/O Specification

Input	644D normalized features $[B, 644]$
Distribution	$644 \to 32$ probability simplex Δ^{32}
References	16 learnable prototypes $\in \Delta^{32}$
Cost matrix	Learnable PSD: $\boldsymbol{M}^\top \boldsymbol{M} \in \mathbb{R}^{32\times 32}$
Sinkhorn	10 iterations, log-domain, $\varepsilon = 0.1$
Output	64D expert representation for PLE gate

Table 13: Optimal Transport Expert I/O specification.

7.6 Implementation Notes

- Cuturi (2013) uses a fixed cost matrix and single target, but our implementation uses a learnable PSD cost matrix and 16 learnable prototypes.
- Sinkhorn iteration count is fixed at 10 to make computation cost predictable during the training loop.
- Synergy with Causal Expert: DeepFM extracts symmetric feature interactions $\langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle$, Causal extracts asymmetric directional causality $W^2_{i,j}$, OT extracts distance functions (metric) $W(\mu, \nu_k)$. The three experts extract *mathematically completely different* structures from the same 644D input.

Key References: Cuturi (NeurIPS 2013), Kantorovich (1942).

8 adaTT (Adaptive Task-aware Transfer)

8.1 Motivation: Negative Transfer in Multi-Task Learning

When 18 simultaneous tasks share expert parameters, gradient conflicts cause negative transfer. Three fundamental limitations of fixed-tower MTL: (1) The shared backbone equally affects all tasks — no mechanism to detect/prevent one task's optimization from degrading another's predictions, (2) Unable to measure which of the 18 task pairs help or harm each other, (3) Fixed weights cannot track task relationships that change across training stages.

8.2 Core Mechanism: Gradient Cosine Similarity

$$\cos(\theta_{i,j}) = \frac{\boldsymbol{g}_i \cdot \boldsymbol{g}_j}{\|\boldsymbol{g}_i\| \cdot \|\boldsymbol{g}_j\|} \tag{45}$$

Where $\boldsymbol{g}_i = \nabla_\theta \mathcal{L}_i$ is the gradient of shared expert parameters with respect to task i's loss.

Why cosine (not Euclidean): (1) Scale-invariant — compares only direction even when task losses differ by orders of magnitude, (2) Interpretable range $[-1, 1] \rightarrow$ maps directly to positive/negative transfer, (3) Efficient computation: after normalization, a single matrix product $\hat{\boldsymbol{G}}\hat{\boldsymbol{G}}^\top$ computes all n^2 similarities.

8.2.1 EMA Stabilization

$$\boldsymbol{A}_t = \alpha \cdot \boldsymbol{A}_{t-1} + (1 - \alpha) \cdot \cos(\theta_t) \tag{46}$$

$\alpha = 0.9$ (effective window ≈ 10 observations). Equivalent to a 1st-order IIR low-pass filter, removing high-frequency batch noise while preserving true task relationship trends.

8.2.2 Transfer-Enhanced Loss

$$\mathcal{L}_i^{\text{adaTT}} = \mathcal{L}_i + \lambda \cdot \sum_{j \neq i} w_{i \rightarrow j} \cdot \mathcal{L}_j \tag{47}$$

$\lambda = 0.1$ (10% influence from other tasks), `max_transfer_ratio = 0.5` (transfer loss cannot exceed 50% of the original loss).

Gradient effect:

$$\nabla_\theta \mathcal{L}_i^{\text{adaTT}} = \nabla_\theta \mathcal{L}_i + \lambda \sum_{j \neq i} w_{i \rightarrow j} \nabla_\theta \mathcal{L}_j \tag{48}$$

The second term is a correction vector that adjusts shared parameters toward directions beneficial to multiple tasks.

8.2.3 Transfer Weight Computation (4-stage)

$$\boldsymbol{R} = (\boldsymbol{W} + \boldsymbol{A}) \cdot (1 - r) + \boldsymbol{P} \cdot r \tag{49}$$

$$\boldsymbol{R}_{i,j} \leftarrow 0 \quad \text{if } \boldsymbol{A}_{i,j} < \tau_{\text{neg}} \tag{50}$$

$$\boldsymbol{R}_{i,i} = 0 \tag{51}$$

$$w_{i \rightarrow j} = \text{softmax}\left(\frac{\boldsymbol{R}_{i,j}}{T}\right) \tag{52}$$

- $\boldsymbol{W}$: Learnable transfer weights (`nn.Parameter`, initialized to 0)
- $\boldsymbol{A}$: EMA affinity matrix
- $\boldsymbol{P}$: Group Prior matrix (domain knowledge)

- r: Prior blend ratio ($0.5 \to 0.1$ via annealing)
- $\tau_{\text{neg}} = -0.1$: Negative transfer threshold
- $T = 1.0$: Softmax temperature

8.2.4 Group Prior

Group	Tasks	Intra-strength	Business Meaning
engagement	has_nba, engagement_score, cross_sell_count, will_acquire_deposits, will_acquire_investments, will_acquire_accounts, will_acquire_lending, will_acquire_payments	0.8	Customer engagement/conversion
lifecycle	churn_signal, product_stability, tenure_stage, segment_prediction	0.7	Customer lifecycle
value	income_tier, spend_level, nba_primary	0.6	Customer value/behavior
consumption	next_mcc, mcc_diversity_trend, top_mcc_shift	0.7	Consumption pattern

Table 14: adaTT task group definitions. Inter-group strength: 0.3.

Prior Blend Annealing (Bayesian interpretation):

$$r(e) = r_{\text{start}} - (r_{\text{start}} - r_{\text{end}}) \cdot \min\left(\frac{e - e_{\text{warmup}}}{e_{\text{freeze}} - e_{\text{warmup}}}, 1.0\right) \tag{53}$$

$r : 0.5 \to 0.1$ implements a prior-to-posterior transition: early training relies on domain knowledge (prior), later training trusts observed gradient affinity (likelihood).

8.2.5 3-Phase Schedule

Phase	Period	Behavior	Purpose
Warmup	Epoch 0 – warmup	Compute affinity only, no transfer loss	Accumulate stable affinity data
Dynamic	warmup – freeze	Active transfer + annealing prior	Learn and apply task relationships
Frozen	freeze – end	Fixed transfer weights (detached)	Fine-tuning stabilization, remove gradient overhead

Table 15: adaTT 3-phase training schedule.

8.2.6 Negative Transfer Detection

$$\boldsymbol{R}_{i,j} = \begin{cases} \boldsymbol{R}_{i,j} & \text{if } \boldsymbol{A}_{i,j} > \tau_{\text{neg}} \\ 0 & \text{otherwise} \end{cases} \tag{54}$$

$\tau_{\text{neg}} = -0.1$ (not 0) allows weakly negative correlations (possible noise) while blocking clearly adversarial gradients.

8.3 Analogy with Transformer Self-Attention

Role	Transformer	adaTT
Query	Current token's query	Current task's gradient direction
Key	Other tokens' response	Other tasks' gradient directions
Similarity	$Q\frac{K^\top}{\sqrt{d_k}}$	Gradient cosine similarity
Value	Other tokens' information	Other tasks' loss values
Output	Weighted context	Transfer loss

Table 16: Structural analogy between adaTT and Transformer self-attention.

8.4 Alternative Comparison

Method	Mechanism	Limitation vs. adaTT
Fixed Weighting	Manual task weights	Cannot measure dynamic affinity
GradNorm	Gradient magnitude balancing	Uses only magnitude, not direction
PCGrad	Project conflicting gradients	Cannot selectively transfer positive knowledge
Nash-MTL	Nash bargaining for Pareto	Computation cost ($O(n^2d)$ vs optimization)
CAGrad	Worst-case gradient alignment	Lacks separation (modularity) of measurement and application

Table 17: MTL approach comparison.

8.5 Implementation Notes

- **Phase 1 (Shared Expert Pretrain):** adaTT active — gradient extraction + transfer loss for 15 epochs.
- **Phase 2 (Cluster Finetune):** adaTT disabled — Shared experts frozen, only cluster-specific sub-heads trained for 8 epochs.
- A lightweight variant of Hypernetworks (Ha et al., 2017): uses observed gradients instead of learned task embeddings as the conditioning signal, enabling zero-delay adaptation to changing task relationships.
- `detect_negative_transfer()` API returns the list of adversarial tasks for each task (e.g., `{"churn_signal": ["has_nba", "engagement_score"]}`).

Key References: Tang et al. (RecSys 2020), Yu et al. (NeurIPS 2020), Fifty et al. (ICML 2021), Chen et al. (ICML 2018), Navon et al. (ICML 2022).

9 Feature Engineering Overview — 11 Disciplines, 316D

9.1 Feature Engineering Philosophy

Traditional statistical features view data through a single lens. Other academic disciplines have developed mathematical tools optimized for specific pattern types over centuries. The key insight is **structural isomorphism** — when mathematical relationship structures are identical regardless of domain objects, the formulas capture the same patterns irrespective of the surface domain.

9.2 Feature Classification by 11 Academic Disciplines

Discipline	Dim	Pattern	Mathematical Tool
Economics (PIH)	8D	Permanent/transitory income decomposition	HP Filter, Kalman Filter
Economics (Micro)	9D	Elasticity, consumption smoothing	Arc elasticity, HHI, Shannon entropy
Chemical Kinetics	6D	Behavioral change velocity/acceleration	Arrhenius equation, finite differences
SIR Epidemiology	5D	Category adoption diffusion	ODE compartmental model
Routine Activity (Criminology)	5D	Regularity, burstiness, temporal anomalies	Circular statistics, burstiness index
Wave Interference	8D	Periodic decomposition, phase synchronization	FFT, Hilbert transform, PLV
TDA (Topology)	70D	Consumption topological structure, phase transition	Persistent Homology, Wasserstein-1
GMM (Statistics)	22D	Soft segmentation, uncertainty	EM algorithm, BIC
HMM (Probabilistic)	48D	Latent state transitions, dynamic trajectories	Forward-Backward, Viterbi, ODE dynamics
MAB (Decision Theory)	4D	Explore/exploit balance	HHI trend, recency-weighted entropy
Graph Embedding	111D	Collaborative filtering + hierarchical structure	LightGCN (64D) + H-GCN (47D)

Table 18: Feature framework derived from 11 academic disciplines. Total 316D (TDA 70D + GMM 22D + HMM 48D + Economics 17D + Multidisciplinary 24D + MAB 4D + Graph 111D + LNN statistics 18D + HMM 5D summary = 316D+ model-derived features).

9.3 Economics Features (17D)

9.3.1 Friedman PIH (8D)

Decomposes observed income into permanent and transitory components:

$$Y_t = Y_t^P + Y_t^T \tag{55}$$

Consumption function: $C_t = k(r, w, u) \cdot Y_t^P$ — consumers spend proportionally to permanent income, while transitory income goes to savings/investment.

Three estimation methods are supported:

- Moving Average: $\hat{Y}_t^P = \frac{1}{L}\sum_{i=0}^{L-1} Y_{t-i}$, $L = 12$

- HP Filter: $\min_\tau \left\{ \sum_t (Y_t - \tau_t)^2 + \lambda \sum_t \left[(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1}) \right]^2 \right\}$, $\lambda = 14400$
- Kalman Filter: $K_{\text{ss}} \approx 0.27$ (73% prior weight, 27% observation weight)

9.3.2 Microeconomic Behavior (9D)

Income Elasticity:

$$\varepsilon_Y = \frac{\partial Q}{\partial Y} \cdot \frac{Y}{Q} = \frac{d \ln Q}{d \ln Y} \tag{56}$$

$\varepsilon_Y > 1$: luxury behavior, $0 < \varepsilon_Y < 1$: necessity, $\varepsilon_Y < 0$: inferior good.

Consumption Smoothing (Hall, 1978):

$$C_t = C_{t-1} + \varepsilon_t \tag{57}$$

Feature: `consumption_smoothing = mu/sigma` (Sharpe ratio analogue for consumption).

Spending Diversification (Shannon Entropy):

$$H = -\sum_{i=1}^{N} s_i \ln(s_i) \tag{58}$$

Category Concentration (HHI):

$$\text{HHI} = \sum_i s_i^2 \tag{59}$$

Shannon entropy and HHI are special cases of Renyi entropy H_α ($\alpha \to 1$: Shannon, $\alpha = 2$: HHI).

9.4 Multidisciplinary Features (24D)

Four modules capture nearly orthogonal projections of the data:
- **Chemical Kinetics:** The *differential structure* of time (1st, 2nd derivatives)
- **Epidemic Diffusion:** *State space transition structure* ($S \to I \to R$)
- **Crime Pattern:** *Statistical texture* of time series (periodicity, clustering, dispersion)
- **Interference:** *Frequency-domain spectral structure* (FFT, coherence, phase)

Cross-module combinations reveal patterns invisible within individual modules: Example: High `catalyst_sensitivity` + high `burstiness` = payday burst spender (optimal target for beginning-of-month promotions).

9.5 GMM Clustering Features (22D)

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{60}$$

$K = 20$ clusters, $D = 40$ input dimensions, full covariance. Output: 20D cluster probabilities γ_{nk} + cluster ID + cluster entropy.

Key advantage of GMM over K-Means: `GroupTaskExpertBasket`'s 20 cluster sub-head outputs are weighted-ensembled using γ_{nk}. Soft assignment improves recommendation quality for boundary customers.

9.6 HMM Triple-Mode Features (48D)

Three parallel Hidden Markov Models:

Mode	States	Time Scale	Target Tasks
Journey (AICRA)	5	Days/weeks	CTR, CVR
Lifecycle	5	Months/years	Churn, Retention
Behavior	6	Monthly patterns	NBA, balance_util

Table 19: HMM Triple-Mode configuration. Each mode 16D = state probs + meta features + ODE dynamics. Total 48D.

ODE Dynamics Bridge (6D per mode): Extracted from Viterbi state trajectory — velocity, acceleration, Lyapunov exponent, cycle period, attractor strength, trajectory length.

9.7 Five-Axis Feature Taxonomy

The full system's 734D main tensor + 68D separate input spans 5 feature axes:

1. **Static/Snapshot:** demographics, account status
2. **Time-series:** Mamba/LNN-derived temporal patterns
3. **Hierarchical:** merchant hierarchy, graph embeddings
4. **Item/Product:** product interaction features
5. **Model-derived:** HMM 5D summary, Bandit 4D, LNN statistics 18D

9.8 Two-Level Ensemble Architecture

The overall architecture is a **two-level ensemble**:

- **Level 1:** Within the Temporal Expert, Mamba/LNN/PatchTST are combined via learned gating
- **Level 2:** Across all 7 Shared Experts (PersLay, DeepFM, Temporal, LightGCN, H-GCN, Causal, OT), CGC Gate Attention performs per-task combination

This hierarchical ensemble ensures both intra-expert diversity (temporal multi-resolution) and inter-expert complementarity (pattern/topology/temporal/relational/causal/distributional).

References

Bauer, U. (2021). Ripser: efficient computation of Vietoris-Rips persistence barcodes. *Journal of Applied and Computational Topology*.

Bello, K. et al. (2022). DAGMA: Learning DAGs via M-matrices and a Log-Determinant Acyclicity Characterization. *ICML*.

Carriere, M. et al. (2020). PersLay: A Neural Network Layer for Persistence Diagrams. *AISTATS*.

Chami, I. et al. (2019). Hyperbolic Graph Convolutional Neural Networks. *NeurIPS*.

Chen, Z. et al. (2018). GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks. *ICML*.

Cohen-Steiner, D., Edelsbrunner, H., & Harer, J. (2007). Stability of Persistence Diagrams. *Discrete & Computational Geometry*.

Cuturi, M. (2013). Sinkhorn Distances: Lightspeed Computation of Optimal Transport. *NeurIPS*.

Dempster, A., Laird, N., & Rubin, D. (1977). Maximum Likelihood from Incomplete Data via the EM Algorithm. *JRSS-B*.

Fifty, C. et al. (2021). Efficiently Identifying Task Groupings for Multi-Task Learning. *NeurIPS*.

Friedman, M. (1957). *A Theory of the Consumption Function*. Princeton UP.

Gu, A. & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *NeurIPS*.

Guo, H. et al. (2017). DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. *IJCAI*.

Ha, D. et al. (2017). HyperNetworks. *ICLR*.

Hall, R. (1978). Stochastic Implications of the Life Cycle-Permanent Income Hypothesis. *JPE*.

Hasani, R. et al. (2021). Liquid Time-constant Networks. *AAAI*.

He, X. et al. (2020). LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. *SIGIR*.

Navon, A. et al. (2022). Multi-Task Learning as a Bargaining Game. *ICML*.

Nickel, M. & Kiela, D. (2017). Poincare Embeddings for Learning Hierarchical Representations. *NeurIPS*.

Nie, Y. et al. (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. *ICLR*.

Rendle, S. (2010). Factorization Machines. *ICDM*.

Rendle, S. et al. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. *UAI*.

Tang, H. et al. (2020). Progressive Layered Extraction: A Novel Multi-Task Learning Model for Personalized Recommendations. *RecSys*.

Wang, R. et al. (2017). Deep & Cross Network for Ad Click Predictions. *KDD*.

Wang, R. et al. (2021). DCN V2: Improved Deep & Cross Network. *WWW*.

Yu, T. et al. (2020). Gradient Surgery for Multi-Task Learning. *NeurIPS*.

Zheng, X. et al. (2018). DAGs with NO TEARS: Continuous Optimization for Structure Learning. *NeurIPS*.